

Retail Sales Analysis

BY NIKHIL KATTAGURI

Project Overview

This project demonstrates practical SQL skills used by data analysts to explore and analyze retail sales data. The analysis is built around business questions rather than isolated SQL exercises. It uses MySQL to measure sales performance, transaction behavior, membership patterns, product performance, category and segment revenue contribution, product penetration, and common product combinations.

Objectives

- Analyze total quantity, revenue before discounts and total discount value.
- Measure unique transactions and average products purchased per transaction.
- Analyze transaction revenue distribution using PERCENT_RANK().
- Compare member and non-member transaction share and average transaction revenue.
- Identify top products and top-selling products within segments and categories.
- Calculate revenue percentage splits using window functions.
- Measure transaction penetration for each product.
- Identify the most common combination of three products purchased together.

Project Structure

Section	Purpose
1. Data & Database Setup	Understand the supplied tables and analytical grain.
2. High-Level Sales Analysis	Measure quantity, revenue and discounts.
3. Transaction Analysis	Measure transaction volume, basket breadth, percentiles, discounts and membership.
4. Product Analysis	Analyze product, segment and category performance.
5. Revenue Contribution	Calculate percentage splits using window functions.
6. Product Penetration	Measure the share of transactions containing each product.
7. Basket Analysis	Find the most common three-product combination.

8. Findings

Summarize the results and business value.

9. Conclusion & How to Use

Explain portfolio value and next steps.

1. Data & Database Setup

The project uses the supplied retail sales dataset and supporting product reference tables. The SQL is designed for execution in MySQL Workbench.

Source Tables

Table	Purpose	Key columns
sales	Transaction-level sales data	prod_id, qty, price, discount, member_m, txn_id, start_txn_time
product_details	Product master and business hierarchy	product_id, product_name, category_name, segment_name
product_prices	Product price reference	id, product_id, price
product_hierarchy	Product hierarchy reference	id, parent_id, level_text, level_name

Core Business Calculation

$\text{Revenue} = \text{qty} * \text{price}$

$\text{Discount Value} = \text{qty} * \text{price} * \text{discount} / 100$

Data Exploration

- Sales records analyzed: 15,095.
- Unique transactions: 2,500.
- Distinct products in sales: 12.
- Product attributes are enriched through product_details using prod_id = product_id.
- The supplied analysis script focuses on analytics; it does not delete source records.

2. High-Level Sales Analysis

What was the total quantity sold for all products?

```
SELECT sum(qty) as quantity_sold
FROM sales;
```

Finding: 45,216 units were sold.

What is the total generated revenue for all products before discounts?

```
SELECT sum(qty * price) as Revenue_before_discounts
FROM sales;
```

Finding: \$1,289,453.00 revenue was generated before discounts.

What was the total discount amount for all products?

```
SELECT sum(qty * price * (discount/100)) as total_discount
FROM sales;
```

Finding: \$156,229.14 total discount value was calculated.

High-Level Findings

Metric	Result
Total quantity sold	45,216
Revenue before discounts	\$1,289,453.00
Total discount value	\$156,229.14
Discount as % of revenue	12.12%

3. Transaction Analysis

How many unique transactions were there?

```
SELECT count(distinct txn_id) as unique_transactions
FROM sales;
```

Finding: 2,500 unique transactions.

What is the average unique products purchased in each transaction?

```
WITH CTE1 AS (SELECT txn_id, count(distinct prod_id) AS uniq_products_purchased
FROM sales
GROUP BY txn_id
ORDER BY 2 DESC)
SELECT ROUND(AVG(uniq_products_purchased),2)
FROM CTE1;
```

Finding: 6.04 unique products per transaction on average.

What are the 25th, 50th and 75th percentile values for the revenue per transaction?

```
WITH CTE1 AS(SELECT TXN_ID, SUM(QTY * PRICE) AS REVENUE
FROM SALES
GROUP BY 1),
CTE2 AS(SELECT REVENUE,
PERCENT_RANK() OVER (ORDER BY revenue) AS PRCT_RANK
FROM CTE1)
SELECT MIN(CASE WHEN PRCT_RANK >= 0.25 THEN REVENUE END) AS PERCENTILE_25,
MIN(CASE WHEN PRCT_RANK >= 0.50 THEN REVENUE END) AS PERCENTILE_50,
MIN(CASE WHEN PRCT_RANK >= 0.75 THEN REVENUE END) AS PERCENTILE_75
FROM CTE2;
```

Finding: P25 = \$376.00; P50 = \$510.00; P75 = \$647.00.

What is the average discount value per transaction?

```
WITH CTE1 AS( SELECT txn_id, SUM((qty * price * discount) / 100) AS transaction_discount
FROM sales
GROUP BY txn_id)
SELECT ROUND(AVG(transaction_discount),2) AS Avg_transaction_discount FROM CTE1;
```

Finding: \$62.49 average discount value per transaction.

What is the percentage split of all transactions for members vs non-members?

```
WITH CTE1 AS (SELECT txn_id, member_m, count(*)
FROM sales
GROUP BY txn_id, member_m)
SELECT member_m, ROUND(COUNT(*) / SUM(COUNT(*)) OVER() *100,2) AS percentage_transactions
FROM CTE1
GROUP BY member_m;
```

Finding: f = 39.80%; t = 60.20%

What is the average revenue for member transactions and non-member transactions?

```
WITH transaction_revenue AS (
SELECT
txn_id,
member_m,
SUM(qty * price) AS revenue
FROM sales
GROUP BY txn_id, member_m)
```

```
)
SELECT
    member_m,
    ROUND(AVG(revenue), 2) AS average_revenue
FROM transaction_revenue
GROUP BY member_m;
```

Finding: f = \$515.04; t = \$516.27

Transaction Findings

- 2,500 unique transactions were identified from 15,095 sales records.
- Average basket breadth is 6.04 unique products per transaction.
- Average discount value is \$62.49 per transaction.
- The project's PERCENT_RANK() approach returns P25 \$376.00, P50 \$510.00, and P75 \$647.00.
- Member/non-member analysis separates transaction share from transaction value.

4. Product Analysis

What are the top 3 products by total revenue before discount?

```
SELECT product_name, SUM(qty*s.price) AS Revenue
FROM sales s
LEFT JOIN product_details ON prod_id = product_id
GROUP BY 1
ORDER BY 2 DESC
LIMIT 3;
```

What is the total quantity, revenue and discount for each segment?

```
SELECT segment_id, SUM(qty) Quantity, SUM(qty* s.price) Revenue, ROUND(SUM(qty*s.price*discount/100),2)
Discount
FROM product_details
JOIN sales s
ON product_id = prod_id
GROUP BY 1;
```

What is the top selling product for each segment?

```
WITH CTE1 AS(
SELECT pd.segment_name AS Segment_Name,
    pd.product_name AS Product_Name,
    SUM(s.qty) as Total_Quantity,
    RANK() OVER(PARTITION BY pd.segment_name ORDER BY SUM(s.qty) DESC) AS `Rank`
FROM sales s
JOIN product_details pd
ON s.prod_id = pd.product_id
GROUP BY 1,2
)
SELECT *
FROM CTE1
WHERE `Rank` = 1;
```

What is the total quantity, revenue and discount for each category?

```
SELECT pd.category_name,  
       SUM(s.qty) AS Total_Qty,  
       SUM(s.price*s.qty) AS Revenue,  
       ROUND(SUM(s.price*s.qty*discount/100),2) AS Discount  
FROM product_details pd  
JOIN sales s  
ON pd.product_id = s.prod_id  
GROUP BY 1;
```

What is the top selling product for each category?

```
WITH CTE1 AS(  
SELECT pd.category_name,  
       pd.product_name,  
       SUM(s.qty) AS Total_Qty,  
       RANK() OVER(PARTITION BY pd.category_name ORDER BY SUM(s.qty) DESC) AS `Rank`  
FROM product_details pd  
JOIN sales s  
ON pd.product_id = s.prod_id  
GROUP BY 1,2)  
SELECT * FROM CTE1  
WHERE `Rank` = 1;
```

What is the percentage split of revenue by product for each segment?

```
WITH CTE1 AS (  
  SELECT  
    PD.SEGMENT_NAME,  
    PD.PRODUCT_NAME,  
    SUM(S.QTY * S.PRICE) AS REVENUE  
  FROM SALES S  
  JOIN PRODUCT_DETAILS PD  
    ON S.PROD_ID = PD.PRODUCT_ID  
  GROUP BY PD.SEGMENT_NAME, PD.PRODUCT_NAME  
) ,  
CTE2 AS (  
  SELECT  
    SEGMENT_NAME,  
    PRODUCT_NAME,  
    REVENUE,  
    SUM(REVENUE) OVER(PARTITION BY SEGMENT_NAME) AS SEGMENT_REVENUE  
  FROM CTE1  
)  
SELECT  
  SEGMENT_NAME,  
  PRODUCT_NAME,  
  REVENUE,  
  ROUND(REVENUE / SEGMENT_REVENUE * 100, 2) AS REVENUE_PERCENTAGE  
FROM CTE2  
ORDER BY SEGMENT_NAME, REVENUE_PERCENTAGE DESC;
```

What is the percentage split of revenue by segment for each category?

```
WITH CTE1 AS (  
  SELECT
```

```

        PD.SEGMENT_NAME,
        PD.CATEGORY_NAME,
        SUM(S.QTY * S.PRICE) AS REVENUE
FROM SALES S
JOIN PRODUCT_DETAILS PD
    ON S.PROD_ID = PD.PRODUCT_ID
GROUP BY PD.SEGMENT_NAME, PD.CATEGORY_NAME
),
CTE2 AS (
    SELECT
        SEGMENT_NAME,
        CATEGORY_NAME,
        REVENUE,
        SUM(REVENUE) OVER(PARTITION BY CATEGORY_NAME) AS CATEGORY_REVENUE
    FROM CTE1
)
SELECT
    SEGMENT_NAME,
    CATEGORY_NAME,
    REVENUE,
    ROUND(REVENUE / CATEGORY_REVENUE * 100, 2) AS REVENUE_PERCENTAGE
FROM CTE2
ORDER BY SEGMENT_NAME, REVENUE_PERCENTAGE DESC;

```

What is the percentage split of total revenue by category?

```

SELECT pd.category_name,
       SUM(s.price*s.qty) AS Revenue,
       ROUND(
           SUM(s.price*s.qty)
           / SUM(SUM(s.price*s.qty)) OVER() *100
           ,2) AS Percentage
FROM product_details pd
JOIN sales s
    ON pd.product_id = s.prod_id
GROUP BY 1;

```

What is the most common combination of at least 1 quantity of any 3 products in a 1 single transaction?

```

SELECT
    s1.prod_id AS product_1,
    s2.prod_id AS product_2,
    s3.prod_id AS product_3,
    COUNT(DISTINCT s1.txn_id) AS transaction_count
FROM sales s1
JOIN sales s2
    ON s1.txn_id = s2.txn_id
    AND s1.prod_id < s2.prod_id
JOIN sales s3
    ON s1.txn_id = s3.txn_id
    AND s2.prod_id < s3.prod_id
WHERE s1.qty >= 1
    AND s2.qty >= 1
    AND s3.qty >= 1
GROUP BY s1.prod_id, s2.prod_id, s3.prod_id

```

```
ORDER BY transaction_count DESC
LIMIT 1;
```

Top 5 Products by Revenue

Rank	Product	Revenue
1	Blue Polo Shirt - Mens	\$217,683.00
2	Grey Fashion Jacket - Womens	\$209,304.00
3	White Tee Shirt - Mens	\$152,000.00
4	Navy Solid Socks - Mens	\$136,512.00
5	Black Straight Jeans - Womens	\$121,152.00

Segment Performance

Segment	Quantity	Revenue	Discount
Shirt	11,265	\$406,143.00	\$49,594.27
Jacket	11,385	\$366,983.00	\$44,277.46
Socks	11,217	\$307,977.00	\$37,013.44
Jeans	11,349	\$208,350.00	\$25,343.97

Category Performance

Category	Quantity	Revenue	Discount	Revenue Share
Mens	22,482	\$714,120.00	\$86,607.71	55.38%
Womens	22,734	\$575,333.00	\$69,621.43	44.62%

5. Revenue Contribution & Window Functions

The project uses window functions when the calculation needs a group total while keeping each product, segment or category row visible. This is the core technique behind the revenue percentage questions.

What is the percentage split of revenue by product for each segment?

```
WITH CTE1 AS (  
    SELECT  
        PD.SEGMENT_NAME,  
        PD.PRODUCT_NAME,  
        SUM(S.QTY * S.PRICE) AS REVENUE  
    FROM SALES S  
    JOIN PRODUCT_DETAILS PD  
        ON S.PROD_ID = PD.PRODUCT_ID  
    GROUP BY PD.SEGMENT_NAME, PD.PRODUCT_NAME  
)  
CTE2 AS (  
    SELECT  
        SEGMENT_NAME,  
        PRODUCT_NAME,  
        REVENUE,  
        SUM(REVENUE) OVER(PARTITION BY SEGMENT_NAME) AS SEGMENT_REVENUE  
    FROM CTE1  
)  
SELECT  
    SEGMENT_NAME,  
    PRODUCT_NAME,  
    REVENUE,  
    ROUND(REVENUE / SEGMENT_REVENUE * 100, 2) AS REVENUE_PERCENTAGE  
FROM CTE2  
ORDER BY SEGMENT_NAME, REVENUE_PERCENTAGE DESC;
```

What is the percentage split of revenue by segment for each category?

```
WITH CTE1 AS (  
    SELECT  
        PD.SEGMENT_NAME,  
        PD.CATEGORY_NAME,  
        SUM(S.QTY * S.PRICE) AS REVENUE  
    FROM SALES S  
    JOIN PRODUCT_DETAILS PD  
        ON S.PROD_ID = PD.PRODUCT_ID  
    GROUP BY PD.SEGMENT_NAME, PD.CATEGORY_NAME  
)  
CTE2 AS (  
    SELECT  
        SEGMENT_NAME,  
        CATEGORY_NAME,  
        REVENUE,  
        SUM(REVENUE) OVER(PARTITION BY CATEGORY_NAME) AS CATEGORY_REVENUE  
    FROM CTE1  
)  
SELECT  
    SEGMENT_NAME,  
    CATEGORY_NAME,
```

```

    REVENUE,
    ROUND(REVENUE / CATEGORY_REVENUE * 100, 2) AS REVENUE_PERCENTAGE
FROM CTE2
ORDER BY SEGMENT_NAME, REVENUE_PERCENTAGE DESC;

```

What is the percentage split of total revenue by category?

```

SELECT pd.category_name,
       SUM(s.price*s.qty) AS Revenue,
       ROUND(
           SUM(s.price*s.qty)
           / SUM(SUM(s.price*s.qty)) OVER() *100
       ,2) AS Percentage
FROM product_details pd
JOIN sales s
ON pd.product_id = s.prod_id
GROUP BY 1;

```

Key Window Function Pattern

```

SUM(REVENUE) OVER(PARTITION BY SEGMENT_NAME)

-- returns the total revenue for the segment on every product row

REVENUE / SEGMENT_REVENUE * 100

-- converts the product revenue into a percentage of segment revenue

```

Why this is useful

- GROUP BY creates one row per group.
- A window function calculates the group total without removing the detailed rows.
- PARTITION BY defines the comparison group.
- SUM(...) OVER() without PARTITION BY gives the overall total.

6. Product Transaction Penetration

Top Products by Penetration

Product	Transactions	Penetration
Navy Solid Socks - Mens	1281	51.24%
Grey Fashion Jacket - Womens	1275	51.00%
Navy Oversized Jeans - Womens	1274	50.96%
Blue Polo Shirt - Mens	1268	50.72%

White Tee Shirt - Mens	1268	50.72%
Pink Fluro Polkadot Socks - Mens	1258	50.32%
Indigo Rain Jacket - Womens	1250	50.00%
Khaki Suit Jacket - Womens	1247	49.88%
Black Straight Jeans - Womens	1246	49.84%
White Striped Socks - Mens	1243	49.72%

COUNT(DISTINCT txn_id) is required because one transaction may contain multiple rows for a product.

7. Basket Analysis

What is the most common combination of at least 1 quantity of any 3 products in a 1 single transaction?

```
SELECT
    s1.prod_id AS product_1,
    s2.prod_id AS product_2,
    s3.prod_id AS product_3,
    COUNT(DISTINCT s1.txn_id) AS transaction_count
FROM sales s1
JOIN sales s2
    ON s1.txn_id = s2.txn_id
    AND s1.prod_id < s2.prod_id
JOIN sales s3
    ON s1.txn_id = s3.txn_id
    AND s2.prod_id < s3.prod_id
WHERE s1.qty >= 1
    AND s2.qty >= 1
    AND s3.qty >= 1
GROUP BY s1.prod_id, s2.prod_id, s3.prod_id
ORDER BY transaction_count DESC
LIMIT 1;
```

Finding: the most common 3-product combination appeared in 352 transactions.

Combination: White Tee Shirt - Mens + Grey Fashion Jacket - Womens + Teal Button Up Shirt - Mens

Why the < conditions are used

The ordered product-ID conditions create one canonical order for each three-product combination. Without them, the same three products could appear as multiple permutations and be counted repeatedly.

8. Findings

Sales Performance

- Total quantity sold: 45,216.
- Revenue before discounts: \$1,289,453.00.
- Total discount value: \$156,229.14 (12.12% of revenue).

Transaction Insights

- 2,500 unique transactions.
- 6.04 average unique products per transaction.
- \$62.49 average discount per transaction.
- P25/P50/P75 transaction revenue: \$376.00 / \$510.00 / \$647.00.

Product & Category Insights

- Top revenue product: Blue Polo Shirt - Mens (\$217,683.00).
- Highest-revenue category: Mens (55.38% of total revenue).
- Highest-revenue segment: Shirt (\$406,143.00).
- Most common three-product combination: White Tee Shirt - Mens, Grey Fashion Jacket - Womens, and Teal Button Up Shirt - Mens (352 transactions).

Business Value

- Supports category and segment performance comparisons.
- Identifies high-value products and frequently purchased products.
- Provides a foundation for basket and cross-sell analysis.
- Demonstrates SQL techniques directly applicable to Data Analyst roles.
- Creates a clean SQL layer that can be connected to Power BI or Tableau.

9. Conclusion

This Retail Sales Analysis project demonstrates a complete business-question-driven SQL workflow. It starts with basic aggregations and progresses to transaction-level analysis, window functions, ranking, penetration and basket analysis. This project shows how SQL can transform raw transaction records into structured metrics that support product, category, segment and customer-membership decisions.

Project Files

File	Description
BTC_PROJECT.sql	SQL script containing the business questions and analysis queries.
sales.csv	Transaction-level retail sales data.
product_details.csv	Product, category and segment attributes.
product_prices.csv	Product price reference.
product_hierarchy.csv	Product hierarchy reference.

Skills Demonstrated

Skill	Used for
MySQL	All analytical queries
CTEs	Multi-step transaction and ranking analysis
JOINS	Combining sales and product attributes
GROUP BY / Aggregation	Sales, product, segment and category metrics
Window Functions	Percentages, ranking and percentiles
COUNT(DISTINCT)	Transaction and penetration metrics
Self-JOINS	Three-product basket combinations
CASE WHEN	Percentile threshold selection